

Value-Based RL

CS 185/285

Instructor: Sergey Levine
UC Berkeley

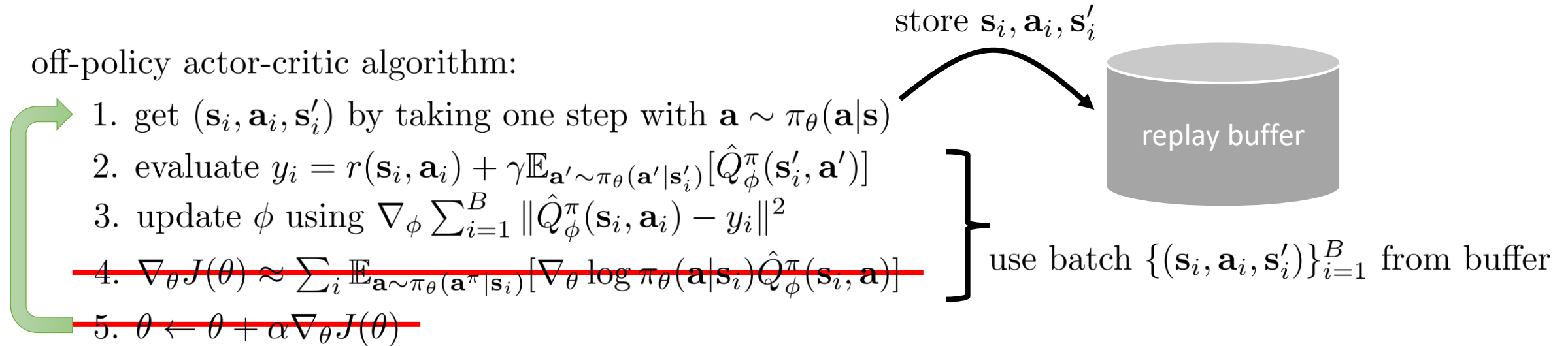


Part 1:

Actor-critic without the actor

Review: actor-critic with a replay buffer

off-policy actor-critic algorithm:

- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]]$
 3. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
 - ~~4. $\nabla_\theta J(\theta) \approx \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)} [\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}_i) \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})]$~~
 - ~~5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$~~
- use batch $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}_{i=1}^B$ from buffer

$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]]$$



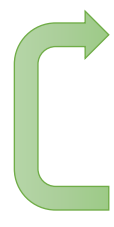
$$r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} \hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')$$

a funny idea:

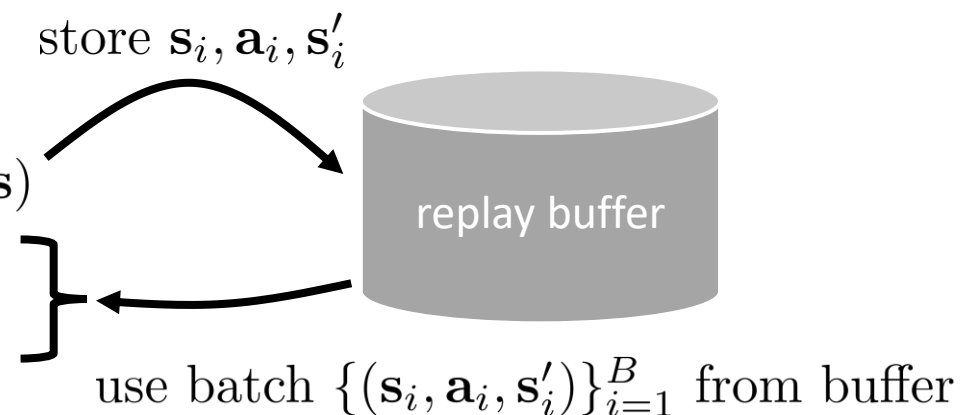
$$\text{set } \pi_\theta(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 & \text{if } \mathbf{a} = \arg \max \hat{Q}_\phi^\pi(\mathbf{s}, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$$

The resulting algorithm

off-policy “critic” algorithm (Q-learning):

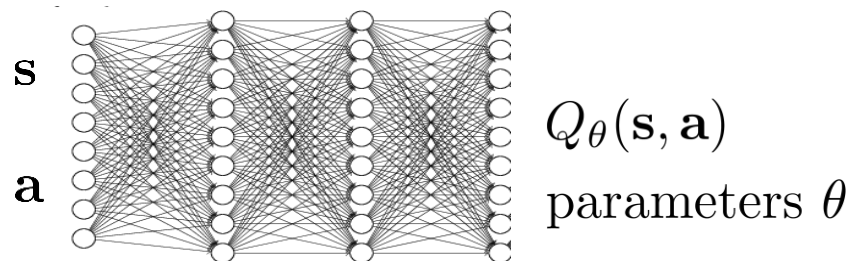
- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')$
 3. update θ using $\nabla_\theta \sum_{i=1}^B \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

$$\pi_\theta(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 & \text{if } \mathbf{a} = \arg \max Q_\theta(\mathbf{s}, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$$



We could just stop here

But it helps to understand more...



Part 2:

Policy iteration and dynamic programming

Policy iteration

High level idea:

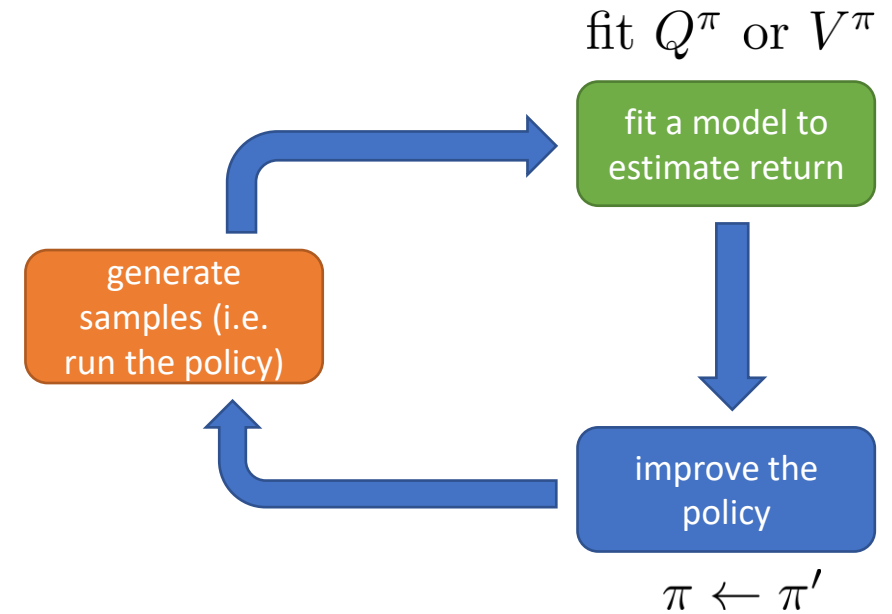
policy iteration algorithm:

- ➡ 1. evaluate $Q^\pi(\mathbf{s}, \mathbf{a})$ ← how to do this?
2. set $\pi \leftarrow \pi'$

$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

as before: $Q^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma E[V^\pi(\mathbf{s}')]$

let's evaluate $V^\pi(\mathbf{s})!$



Dynamic programming

Let's assume we know $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$, and $\mathbf{s}$ and $\mathbf{a}$ are both discrete (and small)

0.2	0.3	0.4	0.3
0.3	0.3	0.5	0.3
0.4	0.4	0.6	0.4
0.5	0.5	0.7	0.5

16 states, 4 actions per state

can store full $V^\pi(\mathbf{s})$ in a table!

$\mathcal{T}$ is $16 \times 16 \times 4$ tensor

bootstrapped update: $V^\pi(\mathbf{s}) \leftarrow E_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})} [r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})} [V^\pi(\mathbf{s}')]]$



just use the current estimate here

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases} \longrightarrow \text{deterministic policy } \pi(\mathbf{s}) = \mathbf{a}$$

simplified: $V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \pi(\mathbf{s}))} [V^\pi(\mathbf{s}')]]$

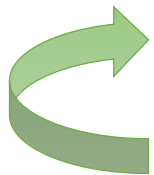
Policy iteration with dynamic programming

policy iteration:

- 
1. evaluate $V^\pi(\mathbf{s})$
 2. set $\pi \leftarrow \pi'$

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

policy evaluation:

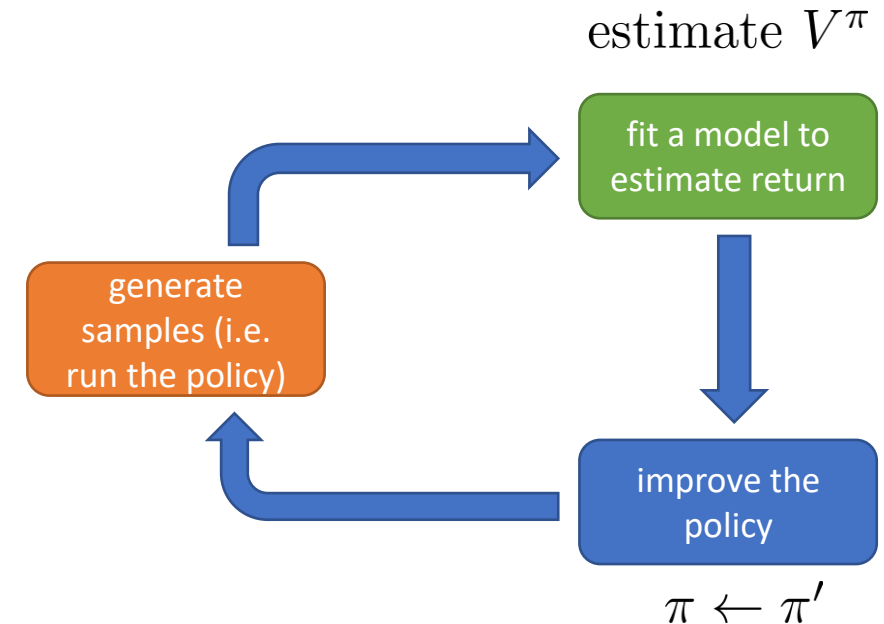

$$V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \pi(\mathbf{s}))} [V^\pi(\mathbf{s}')]]$$

0.2	0.3	0.4	0.3
0.3	0.3	0.5	0.3
0.4	0.4	0.6	0.4
0.5	0.5	0.7	0.5

16 states, 4 actions per state

can store full $V^\pi(\mathbf{s})$ in a table!

$\mathcal{T}$ is $16 \times 16 \times 4$ tensor



Even simpler dynamic programming

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

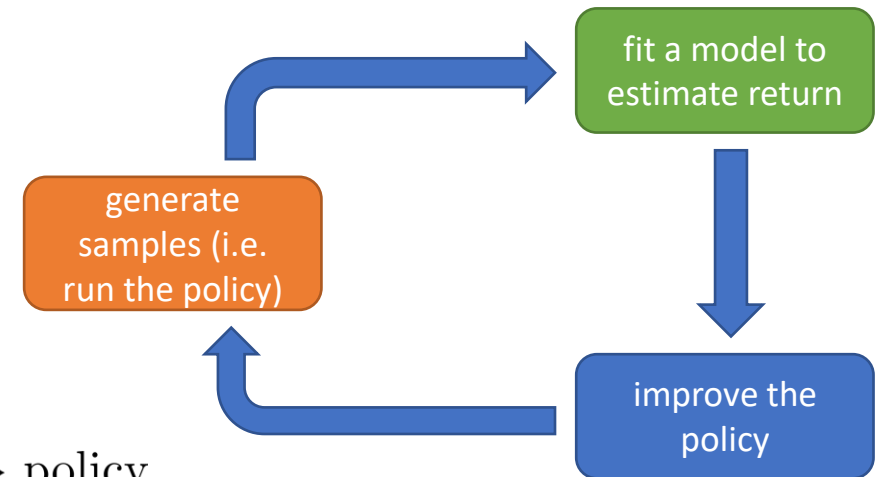
let's evaluate $Q^\pi(\mathbf{s})$!

	a			
s	$Q(\mathbf{s}, \mathbf{a})$		$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$
	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$		$Q(\mathbf{s}, \mathbf{a})$
		$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$
	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$	
	$Q(\mathbf{s}, \mathbf{a})$		$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$
	$Q(\mathbf{s}, \mathbf{a})$	$Q(\mathbf{s}, \mathbf{a})$		$Q(\mathbf{s}, \mathbf{a})$

$\arg \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}) \rightarrow \text{policy}$

approximates the new value!

$$Q^\pi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})} [V^\pi(\mathbf{s}')]]$$



$$V^\pi(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q^\pi(\mathbf{s}, \mathbf{a})$$

skip the policy and compute values directly!

value iteration algorithm:

1. set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')]]$
2. set $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

Part 3:

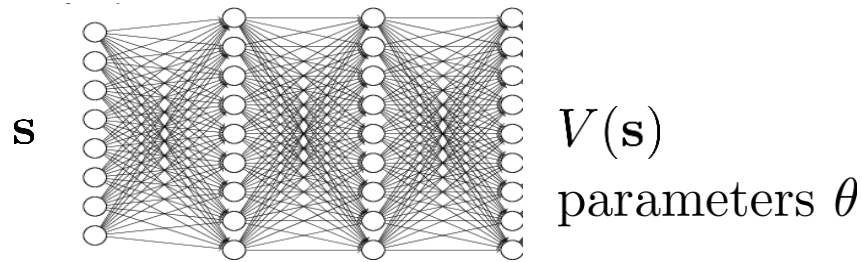
Fitted value iteration

Fitted value iteration

how do we represent $V(\mathbf{s})$?

big table, one entry for each discrete $\mathbf{s}$

neural net function $V : \mathcal{S} \rightarrow \mathbb{R}$



$$\mathcal{L}(\theta) = \frac{1}{2} \left\| V_{\theta}(\mathbf{s}) - \max_{\mathbf{a}} Q^{\pi}(\mathbf{s}, \mathbf{a}) \right\|^2$$

$$\mathbf{s} = 0 : V(\mathbf{s}) = 0.2$$

$$\mathbf{s} = 1 : V(\mathbf{s}) = 0.3$$

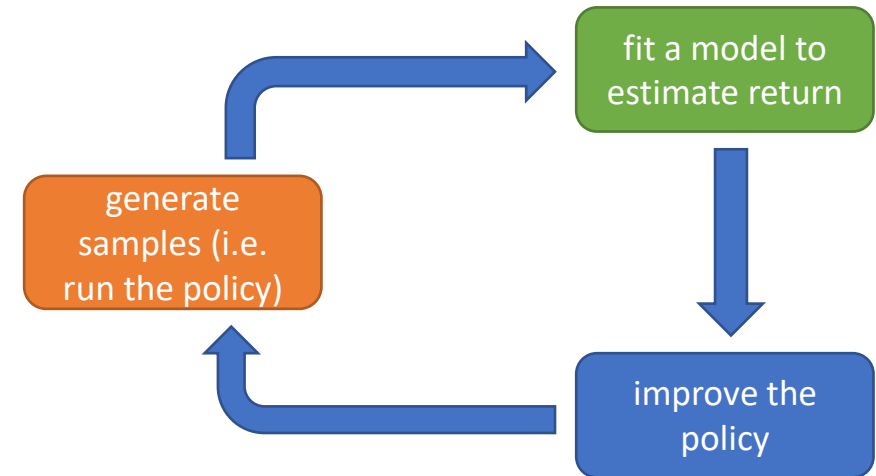
$$\mathbf{s} = 2 : V(\mathbf{s}) = 0.5$$



$$|\mathcal{S}| = (255^3)^{200 \times 200}$$

(more than atoms in the universe)

$$Q^{\pi}(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})} [V^{\pi}(\mathbf{s}')]]$$



$$V^{\pi}(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q^{\pi}(\mathbf{s}, \mathbf{a})$$

fitted value iteration algorithm:

1. set $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_{\theta}(\mathbf{s}'_i)])$
2. set $\theta \leftarrow \arg \min_{\theta} \frac{1}{2} \sum_i \|V_{\theta}(\mathbf{s}_i) - y_i\|^2$

curse of
dimensionality

What if we don't know the transition dynamics?

fitted value iteration algorithm:

- 
1. set $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)])$
 2. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|V_\theta(\mathbf{s}_i) - y_i\|^2$
- need to know outcomes for different actions!

Back to policy iteration...

policy iteration:

- 
1. evaluate $Q^\pi(\mathbf{s}, \mathbf{a})$
 2. set $\pi \leftarrow \pi'$

$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

policy evaluation:


$$V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s}))} [V^\pi(\mathbf{s}')]$$

$$Q^\pi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}' | \mathbf{s}, \mathbf{a})} [Q^\pi(\mathbf{s}', \pi(\mathbf{s}'))]$$

can fit this using samples



Can we do the “max” trick again?

policy iteration:



1. evaluate $V^\pi(\mathbf{s})$
2. set $\pi \leftarrow \pi'$



fitted value iteration algorithm:



1. set $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)])$
2. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|V_\theta(\mathbf{s}_i) - y_i\|^2$

no policy, compute value directly

can we do this with Q-values **also**, without knowing the transitions?

fitted Q iteration algorithm:



1. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\theta(\mathbf{s}'_i)]$
2. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

approximate $E[V(\mathbf{s}'_i)] \approx \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$

doesn't require simulation of actions!

+ works even for off-policy samples

+ only one network, no high-variance policy gradient

- no convergence guarantees for non-linear function approximation (more on this later)

Fitted Q-iteration

full fitted Q-iteration algorithm:

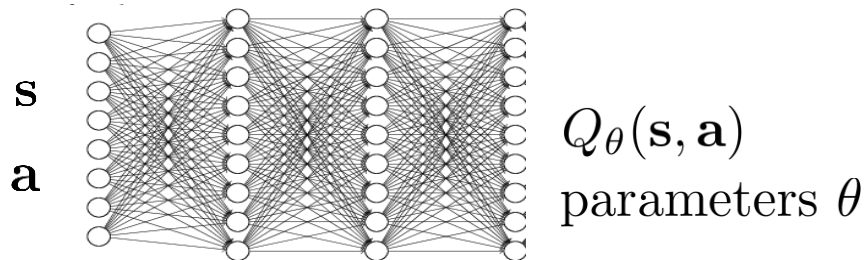
1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
- $K \times$

parameters

dataset size N , collection policy

iterations K

gradient steps S



Part 4:

Back to Q-learning

Why is this algorithm off-policy?

full fitted Q-iteration algorithm:

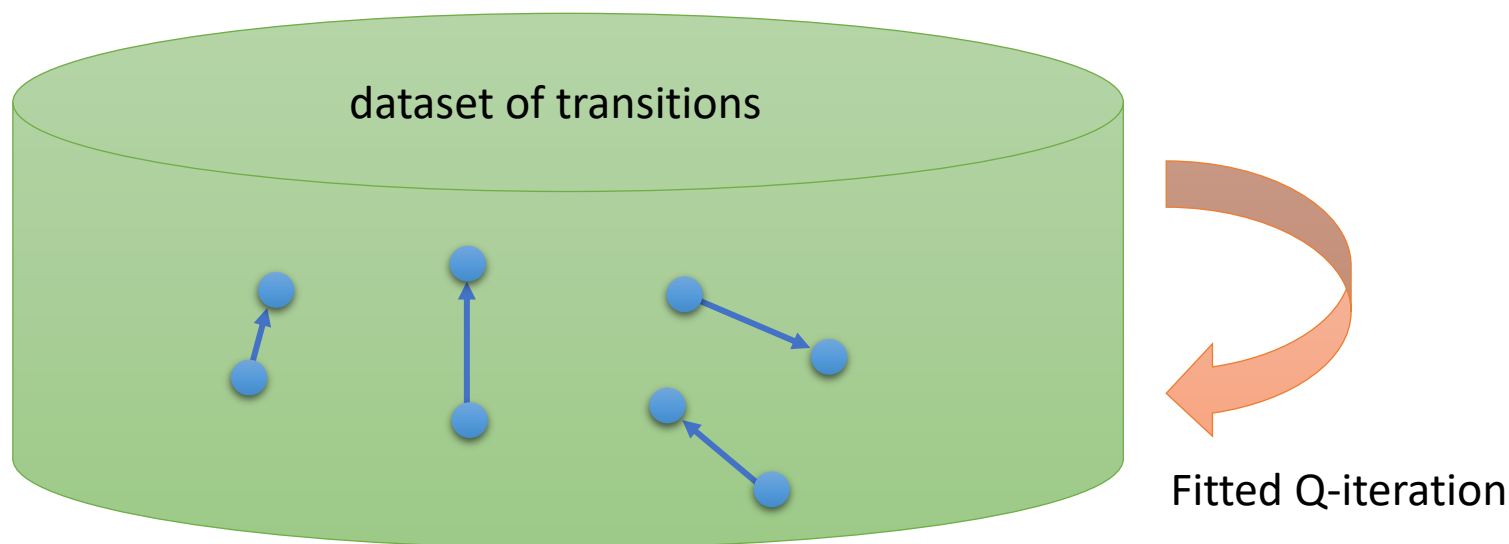
1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

$K \times$

this approximates the value of π' at $\mathbf{s}'_i$

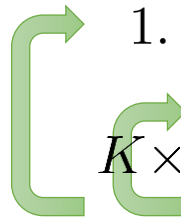
$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

given $\mathbf{s}$ and $\mathbf{a}$, transition is independent of π



What is fitted Q-iteration optimizing?

full fitted Q-iteration algorithm:

- 
1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
 2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$ $\longleftarrow$ this max improves the policy (tabular case)
 3. set $\theta \leftarrow \arg \min_\theta \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
- $\uparrow$
error $\mathcal{E}$

$$\mathcal{E} = \frac{1}{2} E_{(\mathbf{s}, \mathbf{a}) \sim \beta} \left[\left(Q_\theta(\mathbf{s}, \mathbf{a}) - [r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}', \mathbf{a}')] \right)^2 \right]$$

if $\mathcal{E} = 0$, then $Q_\theta(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}', \mathbf{a}')$

this is an *optimal* Q-function, corresponding to optimal policy π' :

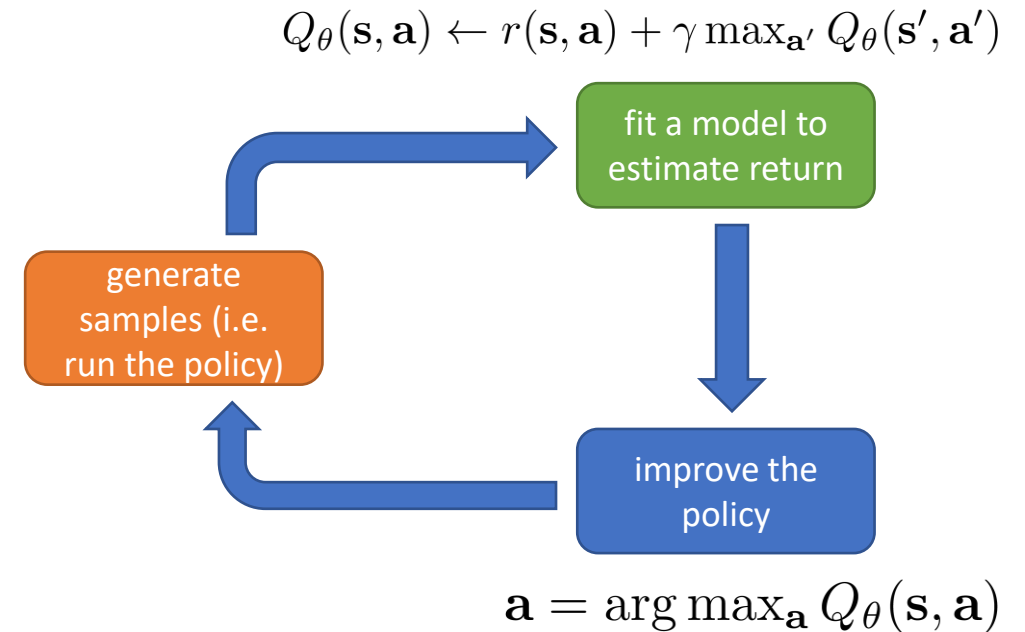
$$\pi'(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q_\phi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases} \quad \begin{array}{l} \text{maximizes reward} \\ \text{sometimes written } Q^* \text{ and } \pi^* \end{array}$$

most guarantees are lost when we leave the tabular case (e.g., use neural networks)

Online Q-learning algorithms

full fitted Q-iteration algorithm:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ using some policy
2. set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\theta \leftarrow \arg \min_{\theta} \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$



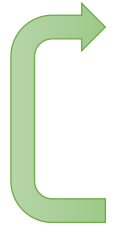
online Q iteration algorithm:

off policy, so many choices here!

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$
2. $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\theta \leftarrow \theta - \alpha \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i)$

Exploration with Q-learning

online Q iteration algorithm:

- 
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$
 2. $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$
 3. $\theta \leftarrow \theta - \alpha \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i)$

final policy:

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q_\theta(\mathbf{s}_t, \mathbf{a}_t) \\ 0 & \text{otherwise} \end{cases}$$

why is this a bad idea for step 1?

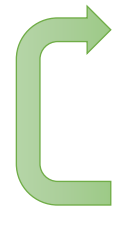
$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \begin{cases} 1 - \epsilon & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q_\theta(\mathbf{s}_t, \mathbf{a}_t) \\ \epsilon / (|\mathcal{A}| - 1) & \text{otherwise} \end{cases} \quad \text{“epsilon-greedy”}$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) \propto \exp(Q_\theta(\mathbf{s}_t, \mathbf{a}_t)) \quad \text{“Boltzmann exploration”}$$

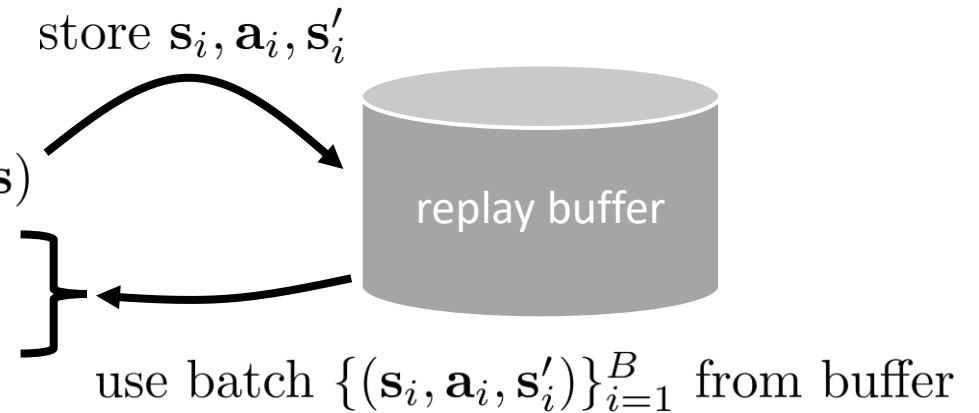
We'll discuss exploration in detail in a later lecture!

Bringing it back together

Q-learning with replay buffer:

- 
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
 2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}')$
 3. update θ using $\nabla_\theta \sum_{i=1}^B \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$

$$\pi_\theta(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 - \epsilon & \text{if } \mathbf{a} = \arg \max_{\mathbf{a}} Q_\theta(\mathbf{s}, \mathbf{a}) \\ \epsilon / (|\mathcal{A}| - 1) & \text{otherwise} \end{cases}$$



If you try to code this up, it almost certainly won't work

We'll find out how to make it work next time!